

```

/**
 * Core Database Access Library
 *
 * Provides a standardized way to log DML, errors, and success states with
 * ANSI color support and environmental filtering.
 *
 * @module QueryUtils
 * @file db/query.util.ts
 *
 * Acknowledgments:
 * Architecture for session-reactive logging and atomic DML utility developed in collaboration
 * with AI on Google Search, powered by the Gemini family of models.
 *
 * @copyright 2018-2026, Dennis Jorgenson
 */

// src/db.ts
import type { ResultSetHeader } from "mysql2/promise";
import type { TOptions, TResponse, TSign } from "#db/types";

import { log } from "#lib/log.util";
import { ApiError, ApiResult } from "#db/types";
import { SC } from "#app/lexicon";

import pool, { DB_SCHEMA } from "#db/db.config";

const module = "Query.Util";

/**
 * Executes DML operations (UPDATE/DELETE) and reports on affected rows.
 *
 * @param {string} sql - The SQL command string.
 * @param {any[]} args - Values to bind to the SQL placeholders.
 * @param {string} context - Traceable context for the operation.
 * @returns {Promise<TResponse>} Standard response indicating success if rows were modified.
 */
const execute = async <T>(sql: string, args: Array<any>, options: TOptions<T>, context: string): Promise<TResponse> => {
  try {
    const { ignore = false, connection } = options;
    const [results] = connection ? await connection.execute(sql, args) : await pool.execute(sql, args);
    const { affectedRows } = results as ResultSetHeader;

    const state = affectedRows > 0 ? `success` : ignore ? `exists` : `no_update`;
    const code = affectedRows > 0 ? SC.OK : ignore ? SC.KEY_EXISTS : SC.NO_UPDATE;
    const action = connection ? `${context}: Transaction` : `${context}: Query`;
    const message =
      code === SC.OK
        ? `-> [Info] ${action}: successful`
        : code === SC.KEY_EXISTS
        ? `-> [Warning] ${action}: Key exists`
        : `-> [Error] ${action}: Data not found on update`;

    return ApiResult(affectedRows > 0, context, { state, code, message, rows: affectedRows });
  } catch (error) {
    const norm = ApiError.normalize(context, error, { sql, args });
    return ApiResult(false, context, {
      state: `error`,
      code: norm.code,
      message: norm.msg,
      meta: norm.data,
    });
  }
};

/**
 * Executes a dynamic SELECT * query based on provided property filters.
 *
 * @template T - The interface representing the expected result row.
 * @param props - Filter criteria (mapped to WHERE clauses).
 * @param options - Configuration including table name, joins (suffix), and limits.
 * @param context - Path for error tracing (e.g., 'UserService.Find').
 * @returns {Promise<TResponse<T>>}> Standardized API response containing rows in 'data'.
 */
export const Select = async <T extends object>(props: Partial<T>, options: TOptions<T>, context = module): Promise<TResponse<T>> => {
  context = `${context}.Select`;

  try {
    const { table, ignore = false, suffix, limit, connection, keys: overrides = {} as Partial<Record<keyof T, TSign>> } = options;
    const parts = [`SELECT * FROM ${DB_SCHEMA}.${table}`];
    const filters: string[] = [];
    const args: any[] = [];
    const cols = Object.keys(props);

    // Single Pass: Iterate over the props object
    for (const [key, value] of Object.entries(props)) {
      if (value === undefined) continue;

      // Direct lookup in the 'keys' record
      const sign = (overrides as any)[key] || "=";
      filters.push(`${key} ${sign} ?`);
      args.push(value);
    }

    if (filters.length) parts.push(`WHERE ${filters.join(" AND ")}`);
    if (suffix) parts.push(suffix);
    if (limit) parts.push(`LIMIT ${limit}`);

    const sql = parts.join(" ").replace(/s+/g, " ").trim();

    // Log the intent before execution
    const logMeta = {
      isTransaction: !!options.connection,
      connectionId: options.connection?.threadId || "pool-default",
    };
    log.dml.select(context, sql, cols, args, logMeta);

    const [results] = connection ? await connection.execute(sql, args) : await pool.execute(sql, args);
    const rows = Array.isArray(results) ? (results as T[]) : [];

    // Logic Gate: Differentiate between empty sets and "not found"
    const hasData = rows.length > 0;
    const isFiltered = args.length > 0;

    const success = hasData || !isFiltered;
    const message = hasData ? "Query successful" : "No matching records found";
    const code = hasData ? SC.OK : isFiltered && ignore ? SC.KEY_NOT_FOUND : SC.NOT_FOUND;

    return ApiResult(success, context, {
      state: "success",
      data: rows,
    });
  }
};

```

```

        message,
        code,
    });
} catch (error) {
const norm = ApiError.normalize(context, error, props);
return ApiResult(false, context, {
    state: "error",
    code: norm.code,
    message: norm.msg,
    meta: norm.data,
});
}
};

/**
 * Dynamically builds an UPDATE statement based on provided keys and properties.
 * Separates filter keys from data columns using splitKeys.
 *
 * @template T - The interface matching the table schema.
 * @param props - Filter criteria (mapped to WHERE clauses).
 * @param options - Configuration including table name, joins (suffix), and limits.
 * @param context - Path for error tracing (e.g., 'UserService.Find').
 * @returns Promise<TResponse> - Success status (fails if no rows match or no columns changed).
 */
export const Update = async <T>(props: Partial<T>, options: TOptions<T>, context = module): Promise<TResponse> => {
    context = `${context}.Update`;

    // Default keys to an empty object if not provided
    const { table, suffix, limit, keys = {} } as Partial<Record<keyof T, TSign>> } = options;

    const sets: string[] = [];
    const filters: string[] = [];
    const setArgs: any[] = [];
    const filterArgs: any[] = [];

    // Single Pass: Iterate over the props object
    for (const [key, value] of Object.entries(props)) {
        if (value === undefined) continue;

        // Direct lookup in the 'keys' record
        const sign = (keys as any)[key];

        if (sign) {
            // It's a Filter (WHERE clause)
            filters.push(`${key} ${sign} ?`);
            filterArgs.push(value);
        } else {
            // It's a Set (UPDATE clause)
            sets.push(`${key} = ?`);
            setArgs.push(value);
        }
    }

    // Early return if there are no columns to update
    if (!sets.length) {
        return ApiResult(false, context, { code: SC.NO_UPDATE, message: `no_change`, meta: props });
    }

    try {
        // Build the SQL parts array for efficiency
        const parts = [ `UPDATE ${DB_SCHEMA}.${table} SET`, sets.join(", ") ];

        if (filters.length) parts.push("WHERE", filters.join(" AND "));
        if (suffix) parts.push(suffix);
        if (limit) parts.push(`LIMIT ${limit}`);

        const sql = parts.join(" ").replace(/s+/g, " ").trim();
        const logMeta = {
            isTransaction: !!options.connection,
            connectionId: options.connection?.threadId || "pool-default",
        };
        // Log the intent before execution

        log.dml.update(context, sql, [...sets, ...filters], [...setArgs, ...filterArgs], logMeta);

        return await execute<T>(sql, [...setArgs, ...filterArgs], options, context);
    } catch (error) {
        const err = ApiError.normalize(context, error, props);
        return ApiResult(false, context, {
            state: "error",
            code: err.code,
            message: err.msg,
            meta: err.data,
        });
    }
};

/**
 * Dynamically builds and executes a SQL INSERT statement.
 *
 * @template T - The Logical Interface (e.g., IUser).
 * @param props - The data object. Undefined properties are stripped to allow DB DEFAULTS.
 * @param options - Configuration including table name and 'ignore' behavior.
 * @param context - Traceable path for debugging (Default: 'Query.Util').
 */
export const Insert = async <T>(props: Partial<T>, options: TOptions<T>, context = module): Promise<TResponse> => {
    context = `${context}.Insert`;
    const { table, ignore = false } = options;

    // 1. Single-Pass Extraction (Inlined Logic)
    const fields: string[] = [];
    const args: any[] = [];

    for (const [key, value] of Object.entries(props)) {
        if (value === undefined) continue;
        fields.push(key);
        args.push(value);
    }

    // 2. Guard: No defined attributes
    if (fields.length === 0) {
        return ApiResult(false, context, {
            code: 400,
            message: `-> [Warning] ${context}: No valid data provided (all fields undefined)`,
        });
    }

    // 3. SQL Construction
    try {
        const sql = `INSERT${ignore ? " IGNORE " : ""} INTO ${DB_SCHEMA}.${table} ( ${fields.join(", ")} ) VALUES (${Array(args.length).fill(" ? ").join(", ")} `;

```

```

const logMeta = {
  isTransaction: !!options.connection,
  connectionId: options.connection?.threadId || "pool-default",
};

// Log the intent before execution
log.dml.insert(context, sql, fields, args, logMeta);

return await execute<T>(sql, args, options, context);
} catch (error) {
  const err = ApiError.normalize(context, error, props);
  return ApiResult(false, context, {
    code: err.code,
    message: err.msg,
    meta: err.data,
  });
}
};

export const Load = async <T>(props: Array<Partial<T>>, table: string, context = "Query.Utilis"): Promise<any> => {
  context = `${context}.Loader`;

  if (!props?.length) return;

  try {
    const fields = Object.keys(props[0] as object);
    const args = props.map((obj) => fields.map((f) => (obj as any)[f]));
    const sql = `INSERT IGNORE INTO ${DB_SCHEMA}.${table} ( ${fields.join(", ")} ) VALUES ?`;

    const [result] = await pool.query(sql, [args]);
    const rows = (result as ResultSetHeader).affectedRows ?? 0;

    // It's a success if it's "Complete", even if 0 rows changed
    // because IGNORE implies "ensure existence"
    const success = true;
    const code = rows > 0 ? 0 : 201; // 0 for "New", 201 for "Already Exists"

    log?.loader(context, table, code, props.length);

    return ApiResult(success, context, {
      state: "success",
      message: rows > 0 ? `Loader: ${rows} rows inserted` : "Loader: No new records (Exists)",
      code,
      rows,
    });
  } catch (error) {
    const err = ApiError.normalize(context, error, props);
    return ApiResult(false, context, {
      state: "error",
      code: err.code,
      message: err.msg,
      meta: err.data,
    });
  }
};

```